GE VERNOVA

ADMS

16.25.1

Software Development Kit (SDK) Programmer Guide

Copyright

Change History

Date	Change Description
Jan 26, 2023	ADMS 3.16: • Editorial and formatting updates.
Oct 20, 2023	ADMS 16.1.0: • The product version numbering has changed. The prefix '3' is no longer included in the version numbering for ADMS.
Apr 16, 2024	ADMS 16.7.0: • Updated document format and styling.

Contents

1. Introduction

ADMS provides a number of programmatic interfaces using RESTful web services and TCP/IP-based protocols to enable integration with third-party products and enterprise applications. The interfaces also support read-only access to ADMS data. Access to these interfaces is subject to an additional license.

This document provides guidance for using the ADMS programmable interfaces to develop custom applications and service requests. References to coding examples are included in the Software Development Kit (SDK).

The *ADMS System Overview* provides context on how the interfaces can be leveraged to integrate with surrounding enterprise systems.

Interfaces data flow is shown in the following image, where the circle represents the service provider side and the bracket represents the consumer side.

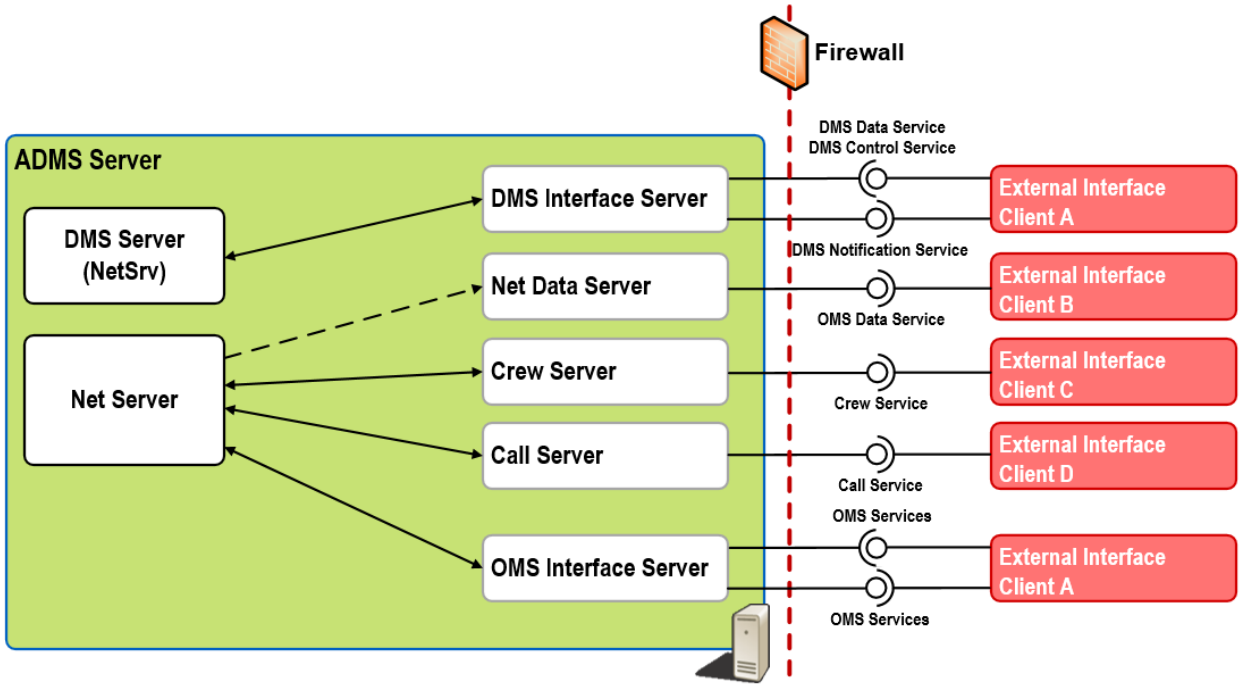


Figure 1. Interfaces

The following table provides an overview of the interfaces that are included in the SDK.

Table 1. Included Interfaces

Interface	Server Name	Protocol	C#	Java	Comments
Call Interface	Call Server	TCP/IP	Yes	No	Payload data available in XML.
Crew Interface	Crew Server	TCP/IP	Yes	No	Payload data available in XML.

Interface	Server Name	Protocol	C#	Java	Comments
Distribution Management Interface	DMS Interface Server	HTTP	Yes	Yes	RESTful approach. Using WCF and ODATA. Payload data available in AtomPub and JSON formats.
Outage Management Read-Only Interface	Net Data Server	HTTP	Yes	Yes	RESTful approach. Using WCF and ODATA. Payload data available in AtomPub and JSON formats.
Outage Management Interface	OMS Interface Server	HTTP	Yes	Yes	RESTful approach. Using WCF and ODATA. Payload data available in AtomPub and JSON formats.

The distribution management and outage management interfaces use standard web service calls, which are based on HTTP. The client (consumer) can use any web client library to send web requests. The SDK provides samples for using these web services in both Java and C#.

The call and crew interfaces use a TCP/IP protocol, which is native to ADMS. Therefore, in order to connect to them, the client needs to use the provided DLLs for connecting to the server and to create and send messages. Related code samples in the SDK are only available in C#.

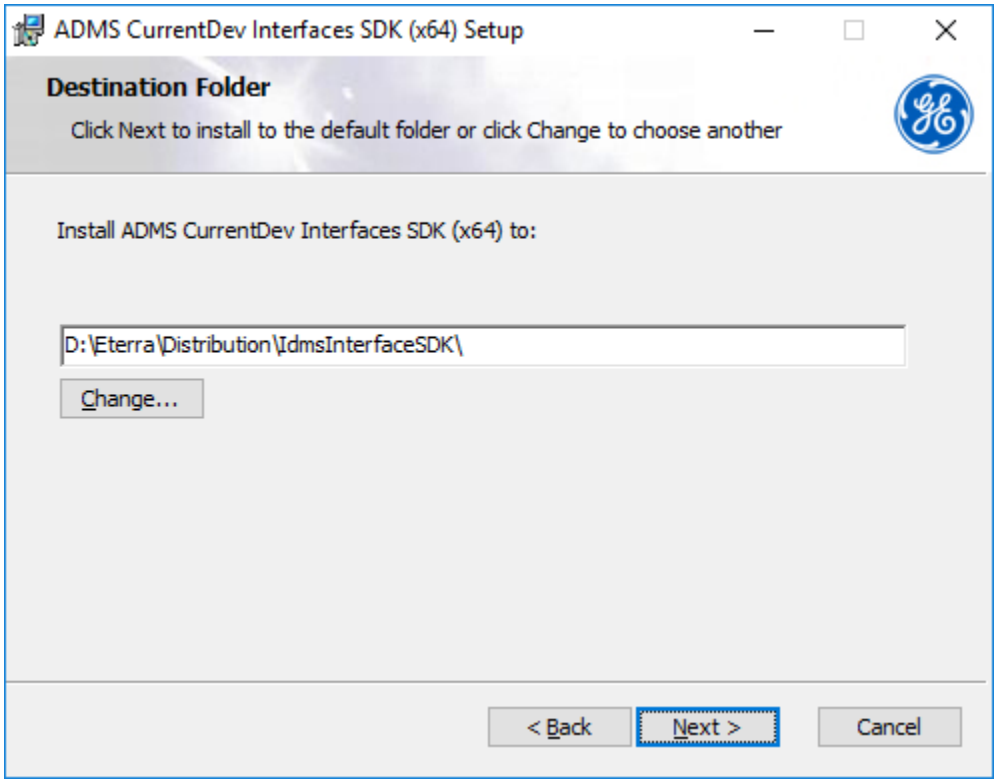
ADMS also provides Notification Publisher, which demonstrates how ADMS web services may communicate with external notification systems. The SDK contains sample code for this application. For more information, refer to Notification Publisher.

2. Installing the Software Development Kit

The SDK contains a set of coding examples and includes the list of DLLs that are needed for running the SDK.

To install the SDK:

1. Run the ADMS_Interfaces_SDK.msi installer.
2. Choose the directory where to install the SDK.



3. When the installation is complete, click Finish.

The installed SDK content is organized in subfolders, as shown in the following image.

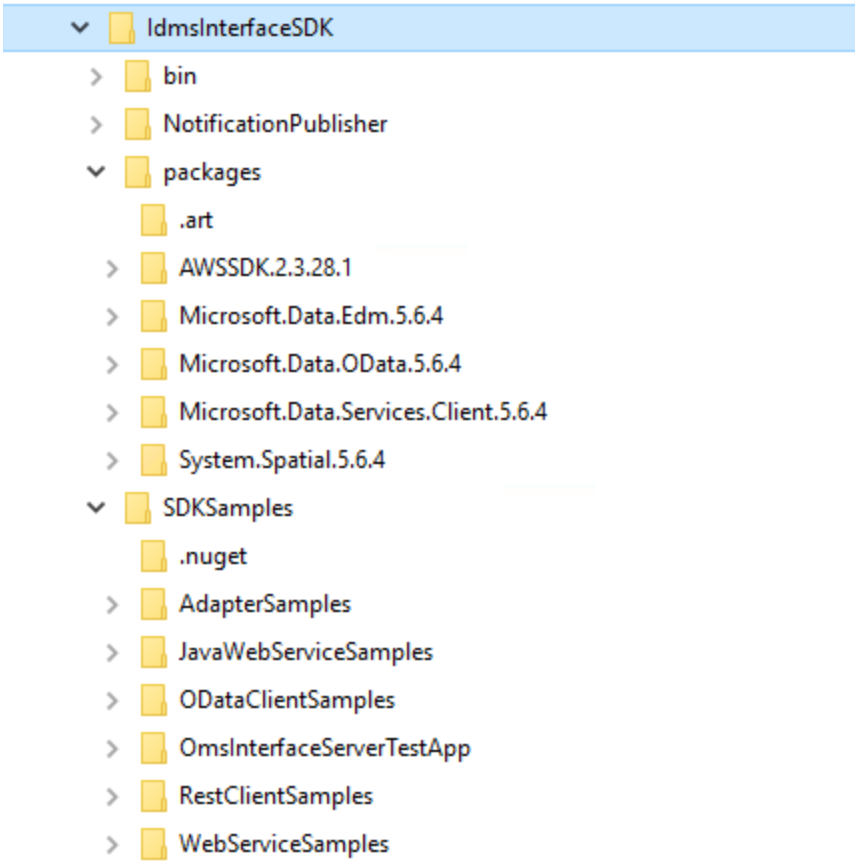


Figure 2. SDK Folders

The SDK content is as follows:

- **bin:** DLLs required for code samples and their prerequisites.
- **NotificationPublisher:** Code samples for a notification publishing application.
- **Packages:** NUPKG files, which are used by NuGet, an extension for Microsoft Visual Studio that provides an interface for managing third-party libraries for .NET projects.
- **AdapterSamples:** Code samples for connecting to Call and Crew servers via TCP/IP.
- **JavaWebServiceSamples:** Code samples for getting DMS and OMS data, sending calls for DMS and SWO requests, and getting notifications for DMS and SWO events.
- **ODataClientSamples:** Code samples for querying DMS and OMS data that are published by DMS Interface Server and Net Data Server.
- **OMSInterfaceServerTestApp:** Sample client for the OMS Interface Server.
- **RestClientSamples:** Code samples for calling web services from DMS Interface Server using .NET ClientBase.
- **WebServiceSamples:** Code samples for calling web services from DMS Interface Server using WebRequest.

3. Call and Crew Interfaces

This chapter describes the SDK samples for the Call and Crew interfaces, which are both TCP/IP interfaces. The coding samples are provided in C#.

Call Server and Crew Server can also run as web services. In web service mode, they support communicating with external applications (WFM and IVR) via HTTP instead of legacy TCP. They can post notifications and send data to external interfaces and allow external applications to access OMS data.

3.1. Overview

ADMS supports communication with external systems, such as Workforce Management System (WFM), Customer Information System (CIS), Interactive Voice Response System (IVR), and Advanced Metering Infrastructure (AMI). This communication is provided by the call server and the crew server. Integration can be created point-to-point, or via an integration bus (Enterprise Service Bus).

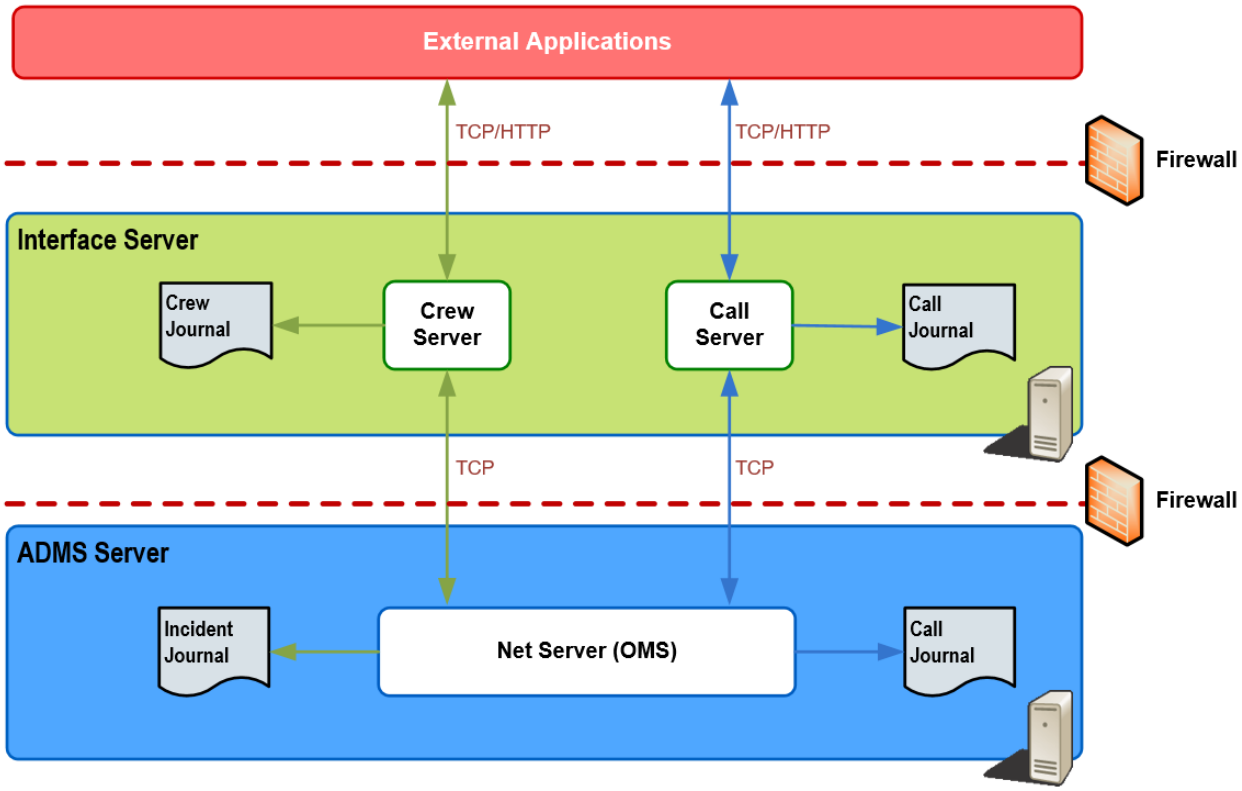


Figure 3. Call and Crew Server Interfaces

The Outage Management interfaces have been implemented using .NET-based technology. They use XML messages for communication and perform XML serialization and de-serialization using .NET-provided classes. Interfaces communicate via TCP/IP or HTML.

Security aspects include using shared key authentication, an authorization header, HTTPS

connections, trusted clients, firewalls, SSL certificates, and message encoding.

For more details about Call Server and Crew Server interfaces, refer to the *ADMS Outage Management Integration Programmer Guide*.

3.2. Code Samples in C#

The SDK includes one sample C# project for Call and Crew interfaces:

- **AdapterSamples:** Demonstrates how to connect to Call and Crew servers via TCP/IP using IdmsConnector as a client.

The AdapterSamples project depends on three DLLs that are delivered with the kit in the \bin\X64\Release directory: NetUtil, OmsUtilities, and IdmsAdapter.

The project includes two main classes:

- CrewAdapter shows how to connect to Crew Server.
- CallAdapter shows how to connect to Call Server.

Both of these classes use the IdmsConnector class from the IdmsAdapter library to connect to the server. To construct IdmsConnector, use one of the constructors below.

```
new IdmsConnector(String[] args);
```

This constructor passes the command line arguments that point to the configuration file (/CONFIG ...\SDKSamples\AdapterSamples\Config\CallAdapterConfig.xml).

```
new IdmsConnector(IdmsAdapterConfig config)
```

This constructs the IdmsAdapterconfig file in the code, and then passes it to IdmsConnector.

After the connector has been created, use the following lines to register for message callbacks, log callbacks, and connection change events:

```
connector.OnLogMessage += new IdmsConnector.LogMessageDelegate(this.LogMessageToConsole);
connector.OnMessageReceivedFromIdms += new
IdmsConnector.MessageReceivedFromIdmsDelagate(this.OnMessageFromIdms);
connector.OnConnectionChangeEventHandler += new
IdmsConnector.OnConnectionChangeDelegate(this.OnConnectionChangeEvent);
```

To send a message to the OMS server:

1. Create a message according to the data that is defined in the NetUtil library.
2. Use the `IdmsConnector.SendMessagetoIdms(msg)` call to send the message.

In the following example, CrewStatusWfmV1 is a message defined in NetUtil. Once you have imported it, you can use it to create and send this type of message to the OMS server. For the details about the server messages, refer to the *ADMS Outage Management Integration Programmer*

Guide.

```
CrewStatusWfmV1 msg = new CrewStatusWfmV1() {
    CrewId = "123",
    Status = 6,  // EnRoute
    Sequence = _sequence++,
    Guid = Guid.NewGuid().ToString()
};
crewConnector.SendMessageToIdms(msg);
```

Use the examples above to configure the server connection in the adapter configuration under the /Config directory and send messages.

3.3. Schema Files

The Swagger and XML Schema Definition (XSD) files are stored in the Schema directory of the SDK. The schema files include schemas for the following applications:

- AMI Server web service
- Call Server web service
- Crew Server Crew Modeling web service
- Crew Server Workforce Management System web service
- ANPS Adapter
- Customer Experience Adapter

Figure 4. Call Server Web Service XSD Schema

4. Distribution and Outage Management Interfaces

This chapter describes the SDK samples for the Distribution Management and Outage Management interfaces. These interfaces are both web service interfaces that use web service calls based on HTTP. The code samples are provided in Java and C#.

4.1. Distribution Management Interface

The Distribution Management interface is supported by the DMS Interface Server, which communicates with the DMS server (NetSrv). This web service interface allows external applications to read and update DMS data such as switch status, device data, DNAF results, switching order data, and annotations. It also allows applications to send requests to a Study Server. DMS Interface Server also supports custom integrations based on its functionality.

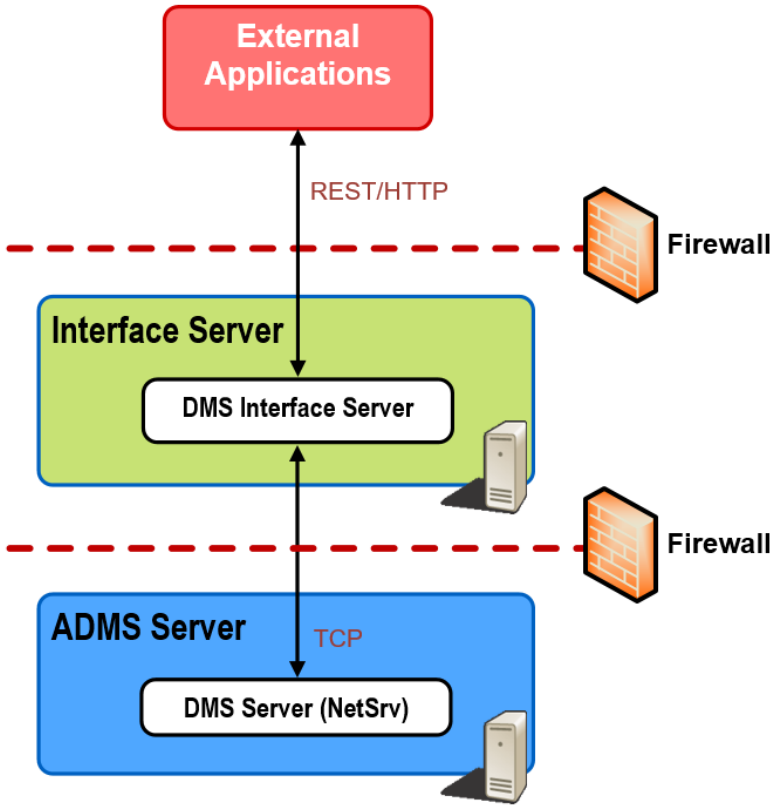


Figure 5. Distribution Management Interface

As shown in the above image, DMS Interface Server runs inside the same firewall as `NetSrv`. It acts as a client to NetSrv, using the same communication protocol that the Browser client uses to talk to the `NetSrv`. Externally, it provides a standard web server interface (REST) to allow third-party applications to communicate with the server from the corporate network and other zones.

The web interface is implemented using Windows Communication Foundation (WCF). DMS Data Collections are specifically provided through the Open Data Protocol (OData). Queried data can be returned in AtomPub or JSON format.

Security aspects include using shared key authentication, trusted clients, firewalls, and HTTPS connections.

For more information, refer to the *Distribution Management Interface Server Programmer Guide.*

4.2. Outage Management Interface

The OMS interface server provides a standard web server interface to allow third-party applications to communicate with the OMS server (NetServer). The OMS interface server acts as a client to the OMS Server, using the same communication protocol that the Browser client uses to communicate with the OMS server. This enables access to the same set of OMS data (for example, incident and crew data) as the client.

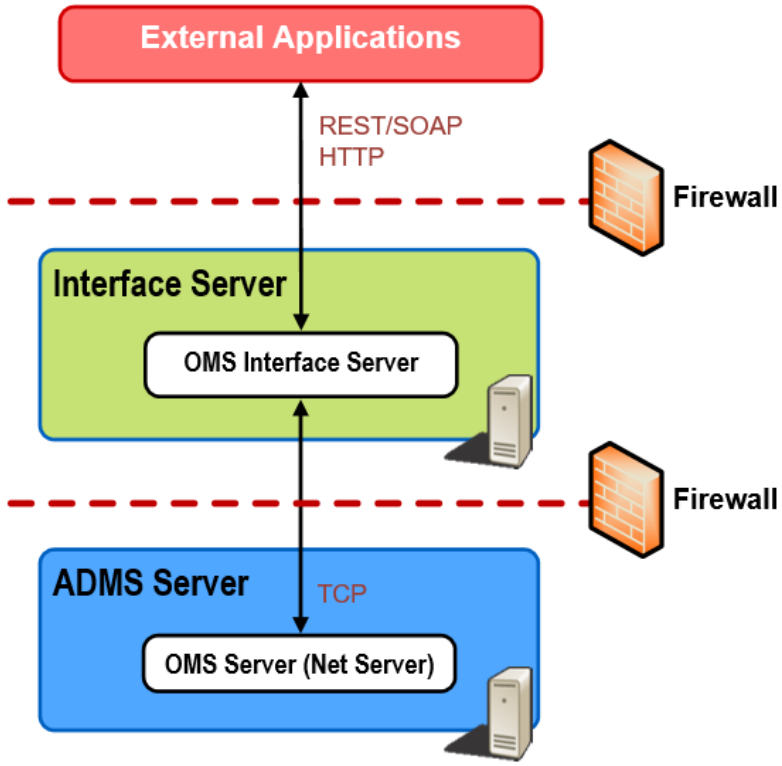


Figure 6. Outage Management Interface

The OMS Services web service is implemented using both SOAP 1.2 and REST.

OMS Data Collections are specifically provided through the Web Services Description Language (WSDL). For details about these standards, refer to https://www.w3.org/TR/soap/ for SOAP and http://schemas.xmlsoap.org/wsdl/ for WSDL.

Open Data Protocol (OData) is used to make it easier for other applications to consume the data.

OData provides a uniform way to expose, structure, query, and manipulate data using REST practices. JSON or ATOM syntax is used to describe the payload. OData also provides a uniform way to represent metadata, allowing applications to know more about the type system, relationships, and structure of the data.

Security aspects include using shared key authentication, trusted clients, firewalls, and HTTPS connections.

For more information, refer to the *Outage Management Interfaces Programmer Guide.*

4.3. Outage Management Read-Only Interface

The outage management read-only interface is supported by the Net Data Server web service interface. It reads snapshots, journals, crew files, and customer models to provide real time outage management data as well as a small window of historical outage management data (usually several weeks).

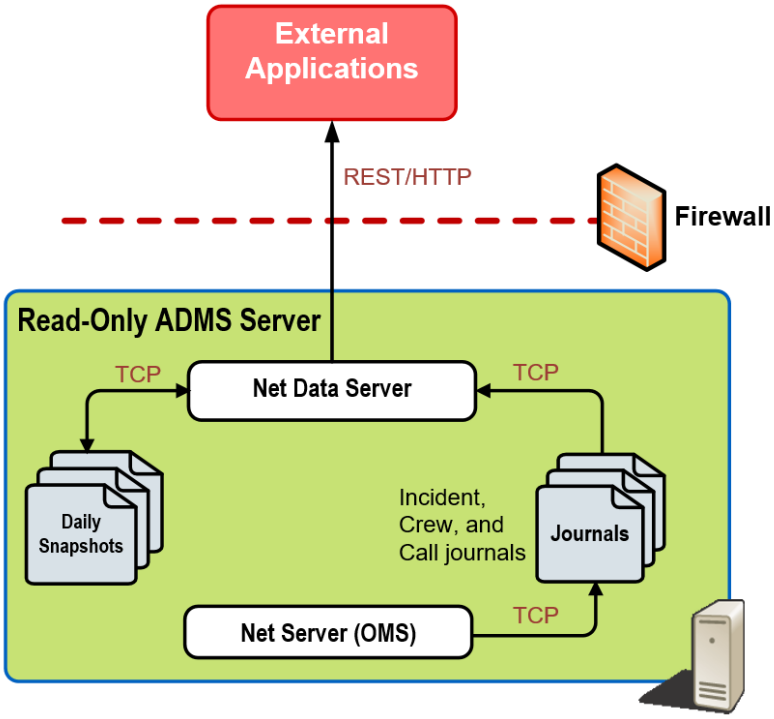


Figure 7. Outage Management Read-Only Interface

As shown in the above image, Net Data Server runs inside the same firewall as Net Server to get journal files from Net Server. To operate in another zone, such as the DMZ, the Net Data Replicator must be configured to supply the journal files to Net Data Server. Externally, Net Data Server provides a standard web server interface (REST) to communicate with third-party applications from the corporate network and other zones.

Interface resources are exposed based on conventions defined in the Open Data Protocol (OData), which is based on representational state transfer (REST) architecture. Queried data can be returned

in AtomPub or JSON format.

Security aspects include using a firewall and HTTPS connection.

For more information, refer to the *Outage Management Read-Only Interface Programmer Guide*.

4.4. Code Samples in C#

The SDK includes three sample C# projects for the DMS Interface Server and Net Data Server web services:

- **ODataClientSamples:** Shows how to query OMS and DMS data that is published in OData.
- **OmsInterfaceServerTestApp:** Shows how to call REST web services from OMS Interface Server using .NET ClientBase.
- **RestClientSamples:** Shows how to call REST web services from DMS Interface Server using .NET ClientBase.
- **WebServiceSamples:** Provides a different method of calling web services from DMS Interface Server. This method uses WebRequest and provides a utility class for this purpose. It also shows how to create a listener for event notifications.
- **NotificationPublisher:** Provides source code for the Notification Publisher application.

4.4.1. ODataClientSamples

This project shows how to query DMS and OMS data that are published by DMS Interface Server and Net Data Server through the OData service.

The project contains two similar classes: DmsDataExample and OmsDataExample. Both classes publish DMS and OMS data via OData, but they use different interface servers and different URLs.

To query DMS and OMS data:

1. Add the service references to your project.

   The URL for OMS data is http://idms-server:8800/Oms, while the URL for DMS data is http://idms-server:8801/dms/data. Replace "idms-server" with the host name where the DMS and OMS servers are running.

2. After the service references have been added, create the data service object. For example:

```
Uri dmsDataUri = new Uri(
   "http://localhost:8801/dms/data/",
   UriKind.Absolute
);
DmsData service = new DmsData(dmsDataUri);
foreach (Station station in service.Stations) {
   // use the station data here
}
```

From the DmsData object, you can get all the collections for published objects (for example, station list and DSO solutions). You can also add a filter to the query (for example, to get only incidents with the PendingClear status).

For more details about data schemas and collections that are available for each of the Data services, refer to the following programmer guides:

- **For OMS data:** *Outage Management Read-Only Interface Programmer Guide*
- **For DMS data:** *Distribution Management Interface Server Programmer Guide*

4.4.2. OmsInterfaceServerTestApp

This project shows how to call the OMS Interface Server web services using .NET ClientBase.

OMS Interface Server provides the OMS Services web service, which is provided through the following URI: http://<Your Server IP>:<Port>/oms/services/. For example, http://localhost:8802/oms/services/.

For the OMS Services web service, the service contract is defined by the OmsServiceClient class. For example, the DmsWebService service contract is defined by the following two classes:

To use the service, there should be a client that acts as a ClientBase for the service contract:

```
class OmsServiceClient : ClientBase<IOmsWebService>, IOmsWebService
{
    private OISConfig config = null;

    public OmsServiceClient(string configFileName)
    {
        config = OISConfig.Deserialize(configFileName);
        if (config != null)
        {
            Endpoint.Address = new EndpointAddress(new Uri(config.WebServiceBaseURL + "services/"));
        }
    }

    public IncidentData GetIncidentById(string id)
    {
        return Channel.GetIncidentById(id);
    }

    public IncidentData GetIncidentByName(string name)
    {
        return Channel.GetIncidentByName(name);
    }

    public void CancelIncident(string id, string userId)
    {
        Channel.CancelIncident(id, userId);
    }
```

```
    public void ClearIncident(string id, string userId)
    {
        Channel.ClearIncident(id, userId);
    }

    public bool UpdateIncident(string id)
    {
        IncidentUpdateData info = new IncidentUpdateData();
        info.IncidentId = id;
        NetDataEntryInfo fld = new NetDataEntryInfo()
        {
        FieldName = "CALLBACKTYPE",
            NewValue = (int)eCallbackType.Automatic,
            OldValue = (int)eCallbackType.Manual
        };
        if (info.UpdateFields == null)
            info.UpdateFields = new List<NetDataEntryInfo>();
        info.UpdateFields.Add(fld);

        if (UpdateIncident(info) == null)
        {
            return false;
        }
        return true;
    }

    public NetDataEntryResponseData UpdateIncident(IncidentUpdateData info)
    {
        return Channel.UpdateIncident(info);
    }
    }
```

Then, for each of the defined methods, call the underlying Channel to pass the call.

To configure the server connection, the project uses the App.config file. For example, the following shows how to configure a connection to DmsWebService that is running on localhost:

```
<configuration>
    <startup>
        <supportedRuntime version="v4.0" sku=".NETFramework,Version=v4.5.2" />
    </startup>
  <system.serviceModel>
    <client>
      <endpoint address="http://localhost:8802/Oms/services/"
                binding="wsHttpBinding"
                contract="OmsInterfaceServer.IOmsWebService" />
      <endpoint address="http://localhost:8802/Oms/services/rest"
          binding="webHttpBinding"
          contract="OmsInterfaceServer.IIncidentManagementService"
          behaviorConfiguration="dmswebservice" />
    </client>
    <behaviors>
      <endpointBehaviors>
```

```
        <behavior name="dmswebservice">
          <webHttp />
        </behavior>
      </endpointBehaviors>
    </behaviors>
  </system.serviceModel>
</configuration>
```

4.4.3. RestClientSamples

This project shows how to call the DMS Interface Server web services using .NET ClientBase.

DMS Interface Server provides the following set of web services:

- **DmsWebService:** /dms/services
- **AnnotationWebService:** /dms/services/annotations
- **ScheduleWebService:** /dms/services/schedules
- **SwoWebService:** /dms/SWO

For each of the web services, the service contract is defined by two classes. For example, the DmsWebService service contract is defined by the following two classes:

- **IDmsWebService:** This interface defines the service contract.
- **DmsWebServiceData:** Defines the data objects that are used in the service.

In order to use the service, you must create a client that will be a ClientBase for the service contract. For example:

```
class DmsWebServiceClient : ClientBase<IDmsWebService>, IDmsWebService
{
    public void UpdateSwitchStatus(SwitchStatus data)
    {
        // pass the call to the underlying channel which acting
        // as a proxy the send the call
        this.Channel.UpdateSwitchStatus(data);
    }
}
```

Then, for each of the defined methods, call the underlying Channel to pass the call.

To configure the server connection, the project uses the App.config file. For example:

```
<endpoint address="http://localhost:8801/dms/services/"
          binding="webHttpBinding"
          contract="RestClientSamples.IDmsWebService"
          behaviorConfiguration="dmswebservice" />
```

The example above shows how to configure a connection to DmsWebService that is running on

localhost. If the server has ServiceAuthCheck turned on, the client needs to send an Authorization header in each request. This is implemented in the HttpDmsMessageInspector class. You need to add the following entries to your App.config and provide the userId and userKey for the client:

```
<behaviors>
  <endpointBehaviors>
    <behavior name="dmswebservice">
      <webHttp />
      <!-- comment the following line if the authentication is not needed -->
      <httpUserAgent userId="DmsISClient" userKey="Test123"/>
    </behavior>
  </endpointBehaviors>
</behaviors>
```

4.4.4. WebServiceSamples

This project shows how to call the DMS Interface Server web services using the WebRequest class. The project includes:

- Sending REST calls to DMS Interface Server for DMS/SWO requests.
- Getting notifications from DMS Interface Server for DMS/SWO events.

Service Example

Since the DMS Interface Server web services use standard REST calls, this sample project shows how to use Windows .NET WebRequest to manually construct the requests and parse the responses.

The WebServiceSamples project provides the following classes:

- **CrewEventListenerExample.cs:** Example class to listen for event notifications from the Crew web service.
- **CrewModelStatusListenerExample.cs:** Example class to listen for status messages from the Crew Modeling web service.
- **CrewModelingExample.cs:** Example class that shows how to send messages to add, update, or incorrectly update a crew; add, update, or delete an employee; and add, update, or delete a vehicle.
- **CrewModelingData.cs:** Example class that shows how to send messages to add, update, or delete crews and add, update, or delete employees.
- **CrewServiceData.cs:** Example class that includes Crew web service data.
- **CrewServiceExample.cs:** Example class that shows how to send crew-related web service calls to Crew Server.
- **DmsEventListenerExample.cs:** Example class to listen for event notifications from DMS Interface Server.
- **DmsServiceData:** Defines the data classes for DMS service requests.

- **DmsServiceExample:** Example class to send service requests to DMS Server.
- **SWOServiceData:** Defines the data classes for SWO service requests.
- **SWOServiceExample:** Example class to send service requests to SWO server (this example has request authentication enabled).
- **WebServiceUtil:** A utility class that uses WebRequest to send service requests. It provides the following helper methods:
  - **SendPostRequest#:** Sends POST web requests.
  - **SendGetRequest:** Sends GET web requests.
  - **SendDeleteRequest:** Sends DELETE web requests.
  - **GetBody:** Formats the request data in either XML or JSON formats.

DmsServiceData and SWOServiceData can be generated from schemas that are published through the interfaces. The schemas are included in the \WebServiceSamples\schema directory for the reference.

To send a POST request:

1. Construct a WebServiceUtil object. If the authentication is needed, use the base URL and userId/userKey.
2. Create the data object for the POST data, and convert the body using the chosen data format.
3. Call SendPostRequest to send the request.

   For example:

```
serviceUtil = new WebServiceUtil(baseServiceUrl);
SwitchStatus swStatus = new SwitchStatus {
    StationName = "Raven",
    SwitchId = "22233858",
    Status = true
};
string reqData = serviceUtil.GetBody(swStatus, typeof(SwitchStatus));
SendPostRequest(swStatusUri, reqData);
```

To send a GET request:

1. Construct a WebServiceUtil object. If authentication is needed, use the base URL and userId/userKey.
2. To send the request, call SendGetRequest.

   For example:

```
serviceUtil = new WebServiceUtil(baseServiceUrl, userId, userKey);
HttpWebResponse resp = serviceUtil.SendGetRequest(uri);
using (StreamReader sr = new StreamReader(resp.GetResponseStream(), Encoding.UTF8)) {
    string result = sr.ReadToEnd();
}
resp.Close();
```

Listener Example

The DmsEventListenerExample class provides an example of how to start a web server and listen for events from DMS Interface Server. It uses the .NET HttpListener class that listens to a given URL. When an event notification comes in, ProcessRequest() is called. You can get the event details from HTTPListenerRequest.

The example below shows using a separate thread for processing the event. For performance reasons, you should consider using ThreadPool in a production system.

```
public DmsEventListenerExample(string listnerUrl)
{
    listener = new HttpListener();
    listener.Prefixes.Add(listnerUrl);
}

public void Start()
{
    listener.Start();
    Console.WriteLine("Listening...");

    while (listener.IsListening) {
        context = listener.GetContext();
        new Thread(this.ProcessRequest).Start();
    }
}

public void Stop()
{
    listener.Stop();
    listener.Close();
}

public void ProcessRequest()
{
    pListenerRequest request = context.Request; // process the request
}
```

4.4.5. Notification Publisher

Notification Publisher is a sample application that demonstrates the use of ADMS web services. It retrieves current OMS data from ADMS, and when configured thresholds are met publishes notifications via the Amazon Simple Notification Service[1] (for more details, refer to Amazon Simple Notification Service).

For example, Notification Publisher allows you to receive email or text (SMS) notifications when significant events occur, such as 1,000 or more customers are de-energized.

Power companies may use Notification Publisher as a template for their own notification applications, which may publish multiple types of notifications, use a different service provider to

push notifications to clients, or send notifications in a specific format for a custom smart phone app (e.g., Android or iOS).

The following diagram shows the Notification Publisher context.

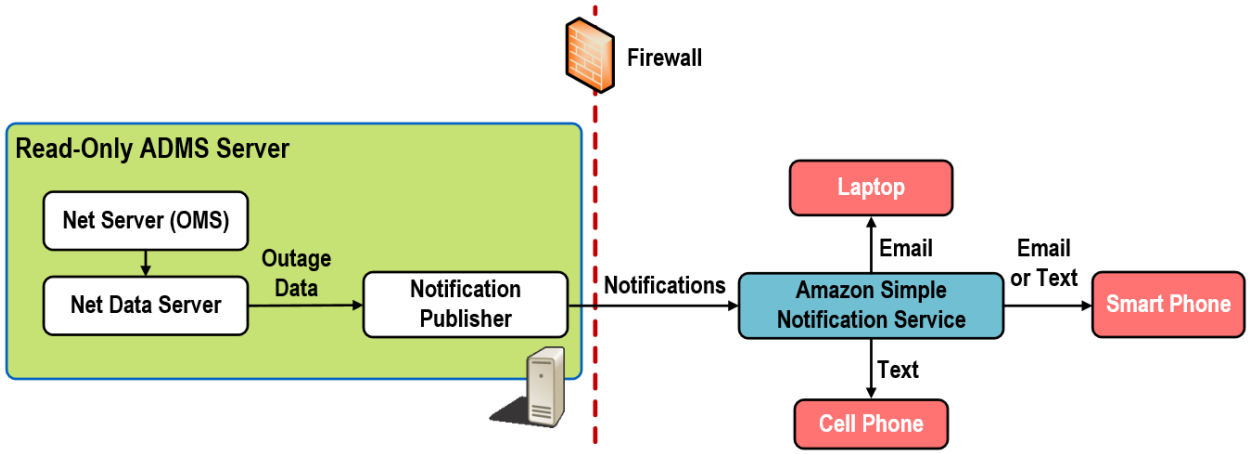


Figure 8. Notification Publisher

Notification Publisher uses the Amazon Web Services (AWS) Software Development Kit (SDK) for .NET[2] to publish notifications to Amazon Simple Notification Service (SNS). AWS SDK for .NET is not included with the Notification Publisher distribution. It must be downloaded separately from Amazon. Once it is downloaded, copy AWSSDK.dll to the same folder as `NotificationPublisher.exe`. For more information, refer to Getting Started with the AWS SDK for .NET[3].

4.4.5.1. Notification Publisher Configuration

Notification Publisher uses the following two configuration files:

- **NotificationPublisherConfig.xml:** This configuration file stores the application settings.
- **NotificationPublisher.exe.config:** This configuration file stores the Amazon Web Service SDK settings.

Notification Publisher is configured by `NotificationPublisherConfig.xml`. This configuration file includes the following parameters:

- **PublishNotifications:** Controls whether any notifications are published. If set to False, Notification Publisher stays idle and does not send notifications.

  This configuration parameter is provided in anticipation of additional configuration parameters that may be added in the future for new notification types. If Notification Publisher is enhanced to include more than one type of notification, there will be one configuration parameter per notification type to enable or disable each notification type.

- **LogFilePathBase:** Specifies the log file path and name.
- **NotificationIntervalMinutes:** Specifies the time period in minutes to wait after a notification is published before publishing another notification. This parameter prevents notifications from

being published too frequently.

By default, Notification Publisher checks for new notifications every 15 seconds. When a notification is published, the next check will be in 15 seconds plus the specified number of minutes.

- **AmazonSnsTopicName:** Specifies the Amazon Simple Notification Service (SNS) topic name for the notification.

  Topics correspond to notification types, and are managed via the AWS Management Console. Subscribers subscribe to each topic for which they need to receive notifications.

  The topic name is the name that appears as the sender of the notification messages. When Notification Publisher publishes a notification, it uses the topic name specified by this configuration parameter, combined with user credentials, to find the "Amazon Resource Number" (ARN) that uniquely identifies the topic, which SNS uses to determine the topic subscribers.

  > **Note**
  >
  > The value of this configuration parameter must correspond to the topic name specified in the AWS Management Console. For Notification Publisher, a single topic named "ADMS" was created; therefore AmazonSnsTopicName must be set to "ADMS". Custom notification applications may have different topic names.

- **OmsDataServiceUriString:** Specifies the Uniform Resource Identifier (URI) of the OMS Data Service exposed by Net Data Server, which is the data source for Notification Publisher. The host name and port number of this URI must match the values specified by the configuration parameters WEBSERVICE_SERVER_NAME and WEBSERVICE_PORT_NUMBER in Net Data Server's configuration file (NetDataServerConfig_localhost.txt).
- **IncidentCountNotificationThreshold:** Specifies the threshold for the number of open incidents. If this threshold is met or exceeded, a notification is published by Notification Publisher.

  For example, if IncidentCountNotificationThreshold = 10, notifications are published if the number of open incidents is greater than or equal to 10.

- **CustomerCountNotificationThreshold:** Specifies the threshold for the number of customers de-energized. If this threshold is met or exceeded, a notification is published by Notification Publisher.
- **CriticalCustomerCountNotificationThreshold:** Specifies the threshold for the number of critical customers de-energized. If this threshold is met or exceeded, a notification is published by Notification Publisher.
- **ListenForDisEvents:** Controls whether the Notification Publisher listens to the DMS interface server. If set to False, Notification Publisher cannot send notifications based on DMS interface server events.
- **DefaultXslTemplateForPdfExport:** Name of the PDF conversion template file for switching

orders.

- **DefaultSubject:** Default subject of the Switching Order notification email. For example, Switch Order Status Changed.
- **DefaultMailBody:** Default body text of the Switching Order notification email.
- **TempFolderPath:** Path to the temporary folder.
- **SMTP:** SMTP settings.
  - **Host:** Host name.
  - **Port:** Port name.
  - **UserName:** User name.
  - **EncryptedPassword:** User password. Use the Encryption Tool to encrypt the password.
  - **UseSSL:** Controls whether SSL is used.
  - **MailFromAddress:** Email from which the notifications are sent.
  - **MailFromDisplayName:** Displayed name of the email from which the notifications are sent. For example, Automatic Switch Order Notifications.
- **ActionSettings:** This group contains the list of notifications.
- **StatusChangedNotificationSettings:** These elements contain settings for particular notifications.
- **ActionName:** Name of a notification event. For example, SWO VALIDATE.
- **XslTemplate:** Name of the PDF conversion template file for this event's switching orders.
- **Subject:** Subject of the notification email for this event. For example, Switch Order was Validated.
- **Body:** Body text of the notification email for this event.
- **DefaultListOfRecipients:** The list of recipients to receive notifications for this event by default. The recipients' emails should be separated with semicolon.

The Amazon Web Services SDK is configured by the `NotificationPublisher.exe.config` configuration file.

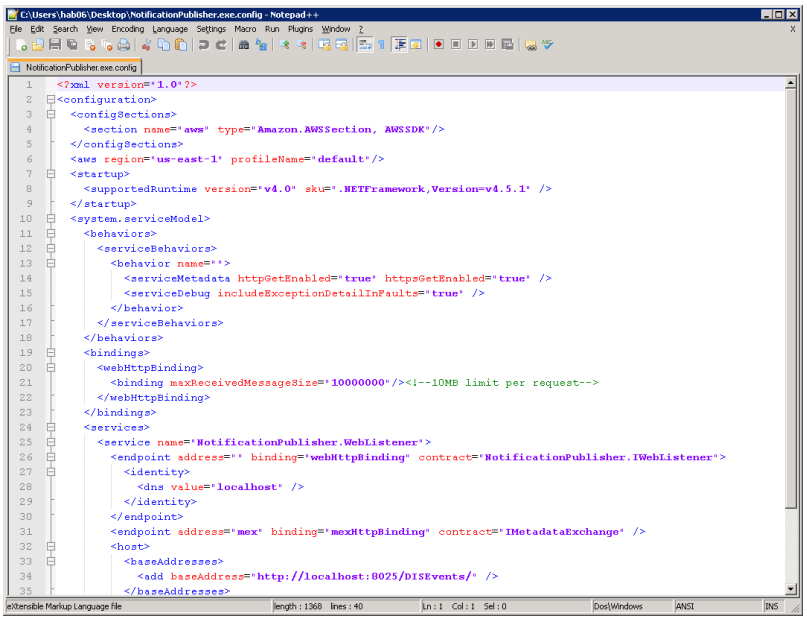


Figure 9. NotificationPublisher.exe.config

This configuration file must be stored in the same folder that contains the Notification Publisher assembly, `NotificationPublisher.exe`. It includes the following parameters:

- **configSections:**
  - **section name:** Amazon Web Services name. Must be set to "aws".
  - **type:** Must be set to "Amazon.AWSSection, AWSSDK"
- **aws region:** Specifies the AWS region. The Notification Publisher region must be set to "us-east-1" (North Virginia), because only this region is currently supported.
- **profileName:** Specifies the AWS Identity and Access Management (IAM) profile that contains the user credentials required to use AWS.
- **startup:**
  - **supportedRuntime version:** Runtime version.
  - **sku:** The version of supported .NET Framework.

This is the name of an account profile created using one of the methods described in the Amazon guide for Configuring AWS Credentials[4]. The default profileName value is "default". In order to use that profile, you must create a profile named "default" on the computer that runs Notification Publisher. You may create a profile with another name as appropriate.

AWS SDK passes the credentials from the specified account profile to SNS when a notification is published. SNS uses those credentials to authenticate the user and verify that the user is authorized to publish notifications.

```
<?xml version="1.0"?>
<configuration>
  <configSections>
    <section name="aws" type="Amazon.AWSSection, AWSSDK"/>
```

```
  </configSections>
  <aws region="us-east-1" profileName="default"/>
  <startup>
    <supportedRuntime version="v4.0" sku=".NETFramework,Version=v4.5.1" />
  </startup>
  <system.serviceModel>
    <behaviors>
      <serviceBehaviors>
        <behavior name="">
          <serviceMetadata httpGetEnabled="true" httpsGetEnabled="true" />
          <serviceDebug includeExceptionDetailInFaults="true" />
        </behavior>
      </serviceBehaviors>
    </behaviors>
    <services>
      <service name="NotificationPublisher.WebListener">
        <endpoint address="" binding="webHttpBinding" contract="NotificationPublisher.IWebListener">
          <identity>
            <dns value="localhost" />
          </identity>
        </endpoint>
        <endpoint address="mex" binding="mexHttpBinding" contract="IMetadataExchange" />
        <host>
          <baseAddresses>
            <add baseAddress="http://localhost:8025/DISEvents/" />
          </baseAddresses>
        </host>
      </service>
    </services>
  </system.serviceModel>
</configuration>
```

4.4.5.2. Running Notification Publisher

Net Data Server must be running to enable Notification Publisher to connect to the OMS Data Service at the URL specified in the configuration file.

The four files necessary to run Notification Publisher are `NotificationPublisher.exe`, `NotificationPublisherConfig.xml`, `NotificationPublisher.exe.config`, and `AWSSDK.dll`. `AWSSDK.dll` should be in the same folder as `NotificationPublisher.exe`, or in another folder that is in the system path so that it can be loaded at run time. `NotificationPublisher.exe.config` must be in the same folder as `NotificationPublisher.exe`. It must specify the AWS region (e.g., "us-east-1"), and the profileName (e.g., "default") that contains the AWS credentials.

To start Notification Publisher, execute a command like the following:

```
NotificationPublisher.exe /CONFIG {NotificationPublisherConfig.xml path}
```

For example, if `NotificationPublisherConfig.xml` is in the same folder as `NotificationPublisher.exe`, the following command will start Notification Publisher:

```
NotificationPublisher.exe /CONFIG .\NotificationPublisherConfig.xml
```

4.5. Code Samples in Java

The Java code samples are stored in the JavaWebServiceSamples directory of the SDK. They perform the following:

- Getting OMS and DMS data that is published through OData.
- Sending REST calls to DMS Interface Server for DMS and SWO requests.
- Getting notifications from DMS Interface Server for DMS and SWO events.

The sample Java code can be built using either Maven or Eclipse. To use Maven, enter the following command, which creates `JavaWebServiceSamples.war`:

```
> mvn clean package
```

Alternatively, you can use the following command to generate an Eclipse project file and run everything from Eclipse:

```
> mvn eclipse:eclipse
```

4.5.1. DMS/OMS Data Samples

As described earlier (see ODataClientSamples), DMS and OMS data are published by DMS Interface Server and Net Data Server using OData. The examples for getting DMS and OMS data are similar; therefore this section describes only querying the DMS data.

The example provided in the SDK uses Restlet, which is a Java framework for REST APIs. It has an extension for handling Odata. For more information about using Restlet, refer to http://restlet.com/technical-resources/restlet-framework/guide/2.3/extensions/odata/overview.

As described in the tutorial, Restlet provides a generation tool to generate entity classes from the published metadata. In the example, this is done by `generateDmsService.bat`. After running the script, it generates all the entity classes under the dmsdatamodel package and the `DmsDataModelService.java` class that provides all the data service. By default, `DmsDataModelService.java` is created in the default package. It needs to be moved to a package, which can be imported and referenced. In the example, it is moved to the dmsdatamodel package with the rest of the entity classes.

To use the data service, you can use the query that is provided by the generated DmsDataModelService and get the data collection. For example, to get a list of stations from DMS, use the following code:

```
dmsData = new DmsDataModelService();
Query<Station> query = dmsData.createStationQuery("/Stations");
```

```
for (Station station : query) {
   System.out.println("Station name: " + station.getName() + ", last DPF run time: " +
station.getDpfExecutionTime());
}
```

OmsDataSample is similar. The only issue is that unlike the DMS data, the OMS entity classes generated by the tool should be refined before use. This is why few issues have been manually cleaned up and added to the omsdataservice package.

4.5.2. WebServiceSamples

Similar to the .NET project, the Java samples show how to make web service requests to DMS Interface Server. They use the RestTemplate class from the Spring framework for REST calls. For more details about using RestTemplate, refer to http://spring.io/guides/gs/consuming-rest/.

The classes are organized as follows:

- **RestClientUtil:** Provides a wrapper to RestTemplate. It has helper methods to construct the request headers and send POST/GET calls.
- **DmsWebServiceSamples:** Provides sample code to send DMS web service requests.
- **SWOWebServiceSamples:** Provides sample code to send SWO web service requests.
- **Domain.dms package:** Provides the data classes for DMS web service calls.
- **Domain.swo package:** Provides the data classes for SWO web service calls.

The domain classes can be generated from the data schemas that are included in the .NET project as samples.

To send a service request:

1. Construct a data object.
2. Call RestClientUtil to send it.

For example:

```
RestClientUtil restClient = new RestClientUtil(“http://localhost:8801);

SwitchStatus aSwitch = new SwitchStatus();
aSwitch.setStationName("PEBBLE BEACH");
aSwitch.setSwitchId("22233858");
aSwitch.setStatus(true);

restClient.sendPostRequest(UPDATE_SWITCH_URI, aSwitch);
```

4.5.3. EventListenerSamples

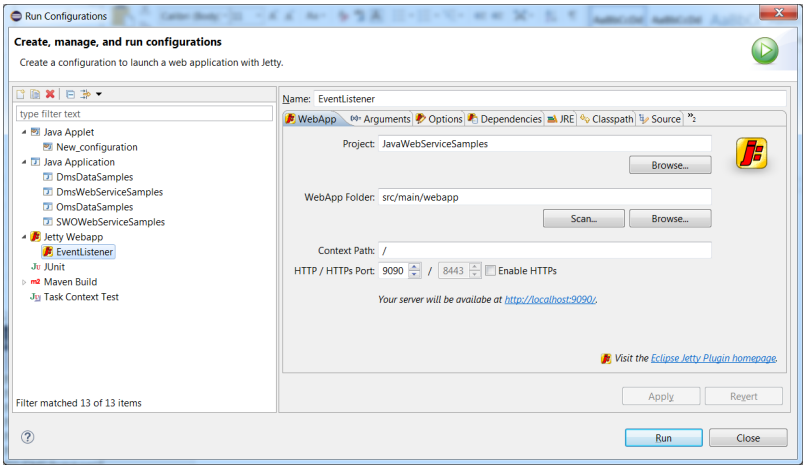
For listening to event notifications from DMS Interface Server, the SDK uses the Spring framework (see http://spring.io/guides/gs/rest-service/).

The notifications are REST calls. Therefore, to map the request, you first need to create a Controller class:

```
@Controller
public class EventController
In the Controller, you can map the events that you are interested in:
@RequestMapping(value = "/event/switchStatus", method = RequestMethod.POST)
@ResponseBody
public String switchStatusEvent(@RequestBody final SwitchEvent event)
{
   logger.info("Receive Switch status Event " + event.toString());
   return "switchStatusEvent Received";
}

@RequestMapping(value = "/event/SWO/StatusNotification", method = RequestMethod.POST)
@ResponseBody
public String swoStatusEvent(@RequestBody final SwitchingOrderStatusNotification event)
{
   logger.info("Receive SWO status Event " + event.toString());
   return "SWO status Received";
}
```

To run the event listener, you can either deploy the war file in a web server, or run the embedded Jetty web server. In Eclipse, to run the Jetty web server, configure the Run Configuration as follows:



To use Maven, run Jetty using the following command:

```
> mvn jetty:run
```

[1] http://aws.amazon.com/sns/

[2] https://aws.amazon.com/sdk-for-net/

[3] http://docs.aws.amazon.com/AWSSdkDocsNET/latest/DeveloperGuide/net-dg-setup.html

[4] http://docs.aws.amazon.com/AWSSdkDocsNET/latest/DeveloperGuide/net-dg-config-creds.html#net-dg-config-creds-sdk-store

5. Amazon Simple Notification Service

Amazon Simple Notification Service (SNS) is a web service that sends notifications to subscribed computers or mobile devices (phones). It uses the Publish/Subscribe architecture pattern, in which the publisher application (for example, Notification Publisher) "publishes" notifications to SNS, and SNS then pushes the notifications to subscribers.

To receive messages, you need to subscribe to SNS. Subscription is a two-step process. New subscriptions may be entered via an Amazon SNS web page either by an administrator or directly by the person who needs to receive the notifications. After subscription, an email or text is sent to the subscriber's email address or phone number. Subscribers must confirm that they want to receive notifications, either by clicking a link in the email or by replying to the text message. Once confirmed, published notifications are pushed to subscribers. Subscribers may unsubscribe at any time.

To use Amazon SNS, you must create your own AWS account and set up user credentials via Amazon SNS or another service provider. A limited number of notifications (1,000,000) may be published to SNS at no charge, but you still must have an account. Once the root account is created, create user accounts for the people and applications (for example, for Notification Publisher) who use SNS.

Security credentials for the Notification Publisher user account must be stored in a "profile" parameter of the `NotificationPublisher.exe.config` configuration file. For more information about security credentials, refer to AWS Security Credentials[1].

[1] http://docs.aws.amazon.com/general/latest/gr/aws-security-credentials.html